

# Project Requirements and Specifications – Industrial Informatics

## 1) General Project Requirements

A **complex software application** will be developed using the **C# programming language** in the **Visual Studio development environment**.

The project topic is chosen by the team, but it must follow the principles of **Object-Oriented Programming (OOP)** and demonstrate the ability to **design and implement a functional software system**.

The application must include:

- a **structured software architecture**;
- a **graphical user interface (GUI)** for the user;
- **persistent data management**;
- implementation of **Object-Oriented Programming concepts**.

## 2) Project Deliverables

During the semester, the following **deliverables** will be developed and evaluated:

### E1 – Project Implementation Plan

A **PDF document** that must include:

- the **team composition**, a short presentation of the members, and their roles;
  - the **selected project topic**;
  - a **description of the proposed software system**;
  - the **implementation plan**;
  - the **task list**;
  - the **activity schedule**.
- 

### E2 – Software Modeling and Design Specifications

A **PDF document** that must include:

#### 1. UML Class Diagram

- the main classes of the application;

- attributes and methods;
- relationships between classes (inheritance, association, aggregation, etc.).

## 2. Application Architecture Description

- modular or layered structure (UI, business logic, data access).

## 3. Graphical User Interface Design

- sketches or mockups of the application windows.

## 4. Description of the Main Classes

- the role of each class;
- responsibilities.

## 5. Test Plan

- definition of test cases;
- input values and expected results.

The deliverable must be submitted in **electronic format** and represents the **foundation for implementing the application during the development phase**.

---

### E3.1 – Software Prototype (Release 1)

The **first functional version of the application**, which must include:

- implementation of the **main functionalities**;
  - an **operational user interface**;
  - implementation of the **core concepts of the application**.
- 

### E3.2 – Improved Software Prototype (Release 2)

This stage is **optional** and can be completed if students wish to **improve their grade**.

The prototype should include:

- **significant improvements** to the application;
- **code optimization**;
- **additional functionalities**;
- improvements to the **graphical user interface**.

---

## E3 – Final Report

The **final report** must include:

- the **technical documentation** of the project;
- description of the **application architecture**;
- description of the **classes and modules**;
- **testing report**;
- **conclusions**.

The team will also present the project through a **PowerPoint presentation** or through an **application workflow demonstration**.

### 3) Team Composition and Roles

For the development of the project, **teams of 4–5 students** will be formed with the following roles:

#### 1. Team Leader

The main role of the **team leader** is to coordinate the entire team and ensure the interface between the team members and the supervising instructor.

The team leader must ensure that each team member has a **clearly defined task for the current week**, as well as all the necessary information required to complete it.

The team leader has the following **rights and responsibilities**:

- to evaluate the activity of each team member in order to ensure the **continuous improvement of the deliverables** produced by the team;
- to modify the internal structure of the team if circumstances require it (illness, travel, etc.). These changes may be **permanent** for the entire duration of the project development, in which case the team leader must notify the supervising instructor. If the change is **temporary**, due to specific circumstances, notification is not required. No team member may be removed from the team, nor may new members be added;
- to read **scientific and technical articles**, conduct **bibliographic research**, and write **technical reports** that will support the software implementation;
- to prepare, draft, and, if necessary, modify the **Project Implementation Plan** during the documentation stage;
- to design the **object-oriented model**, the **architectural design**, and the **functional design** of the application. During these activities, the team leader will be assisted by the programmers and the tester. The team leader will also sketch the **graphical user interface** of the application and define the **data persistence solutions**;
- to create **class skeletons** based on the application specifications and architectural model;

- to discuss **test cases** together with the designated tester during the modeling phase. The tester will develop **unit tests** for the designed classes during the modeling stage;
- to work together with the tester to define **test cases early** (input values and expected outputs) before the programmers start writing the source code;
- to establish **graphical user interface test cases** together with the tester;
- to initiate **technical meetings** with the team members and organize working meetings at the request of any team member;
- to review deliverables and send them back for **correction or revision** if the review results are unsatisfactory;
- to ensure that all team members clearly understand their tasks and have all the necessary materials to perform their work efficiently;
- to establish **internal deadlines** within the team;
- to ensure that team members **respect the established deadlines**;
- to contribute, together with the programmers and the tester, to the **project documentation**;
- to participate in the **scheduled meetings with the supervising instructor**.

The team leader is **directly responsible for the completion of the following deliverables**:

- **Activity reports**
- **E1 – Project Implementation Plan**
- **E2 – Software Modeling and Design Specifications**
- **E3.1 – Software Prototype (Release 1)**
- **E3.2 – Software Prototype (Release 2)** (if students wish to improve their grade; the project must be enhanced and presented again during the retake session)
- **E4 – Final Report**

## 2. Developer

The **developer** is responsible for implementing the **source code of the application**.

### Responsibilities

- implementation of classes and methods;
- development of the application's functionalities;
- adherence to the defined software architecture;
- collaboration with the team leader for the **integration of software components**.

The developer must follow **good programming practices**, such as:

- modularity;
- code reuse;
- proper code documentation.

### 3. Tester

The **tester** is responsible for verifying the **functionality of the application**.

#### Main activities

- defining **test cases**;
- creating **unit tests for classes**;
- testing the implemented functionalities;
- identifying and reporting **software errors (bugs)**.

Testing is performed in **two stages**:

1. **Before implementation** – defining the **unit tests**.
2. **After implementation** – verifying the **functionality of the application**.

Students responsible for testing are encouraged to study **Software Testing concepts and modern testing methods**.

Example of role assignment in a team of 5 students::

| Team Role                       | Main Responsibilities                                                                    | Specific Activities                                                                                                                                                                                 |
|---------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Team Leader                     | Coordinates the team's activities and establishes the technical direction of the project | Plans the project phases, assigns tasks, defines the application architecture, monitors team progress, organizes technical meetings, reviews deliverables, and contributes to project documentation |
| Frontend Developer              | Develops the user interface of the application                                           | Designs and implements the graphical user interface, integrates UI components with the application logic, validates user input, and optimizes the user experience                                   |
| Backend Developer               | Implements the application logic and data access                                         | Develops business logic components, manages database access, implements services and APIs, and integrates the application with other software components                                            |
| Full-Stack Developer (optional) | Contributes to both frontend and backend development                                     | Implements complete application functionalities, integrates the interface with the application logic, and participates in the development and optimization of system components                     |
| Tester (QA Engineer)            | Verifies the functionality of the application and identifies software errors             | Defines test cases, performs unit and functional tests, reports bugs, verifies the correctness of application workflows, and validates the final version                                            |

## 4) Project Schedule

The project is organized into **four main stages**.

### Stage 1 – Analysis

**Deadline: Week 5 (Semester II)** Activități:

**Activities:**

- defining the **system requirements**;
- selecting the **project topic**;
- defining the **overall architecture** of the system.

**Deliverable:**

- **Project Implementation Plan**

### Stage 2 – Modeling

**Deadline: Week 10 (Semester II)**

In this stage, the **detailed design of the application** is carried out based on the requirements defined during the analysis phase.

**Main activities include:**

- developing the **object-oriented model** of the application;
- designing the **class structure and relationships** between classes;
- defining the **application architecture**;
- designing the **graphical user interface**;
- defining **test cases and validation scenarios**.

**Deliverable:**

- **Software Modeling and Design Specifications**

## Stage 3 – First Development Iteration

### Deadline: Week 14

#### Activities:

- implementing the **software prototype**;
- testing the implemented functionalities;
- presenting the **project**.

#### Deliverables:

- **Software Prototype – Release 1**
- **Final Report (project documentation)**

## Stage 4 – Second Development Iteration (Optional)

### Deadline: Retake session

#### Activities:

- improving the **application**;
- implementing **additional functionalities**;
- optimizing the **source code**.

#### Deliverable:

- **Software Prototype – Release 2**

## Requirements for Project Evolution

Between **Release 1** and **Release 2**, there must be **significant improvements**, such as:

- new functionalities;
- performance optimizations;
- improvements to the graphical user interface;
- extension of the application architecture.

## Important Notes

- The project is carried out **in teams**, but the **evaluation is individual**.
- Each student will receive a grade based on their individual contribution to the project.
- The **team leader must submit activity reports every two weeks**.
- Late submission of reports will be **penalized with –0.5 points for each delay**.

